

Concerns about the design of `std::execution::task`

Document #: P3801R0
Date: 2025-07-15
Project: Programming Language C++
Audience: LEWG
Reply-to: Jonathan Müller
<foonathan@jonathanmueller.dev>

Contents

- 1 Abstract
- 2 Background
- 3 Major concerns
 - 3.1 Iterative code can stack overflow
 - 3.2 Parameter lifetime is surprising
 - 3.3 No protection against dangling references
- 4 Minor concerns
 - 4.1 `co_yield with_error(x)` is clunky
 - 4.2 `co_await ex::schedule(sch)` is an expensive no-op
 - 4.3 Coroutine cancellation is ad-hoc
- 5 Next steps
- 6 References

§ 1 Abstract

[P3552R3], proposing the coroutine task type `std::execution::task`, was approved in Sofia for inclusion in C++26. I have strong concerns about its design and urge the committee to reconsider.

§ 2 Background

[P3552R3] (finally) proposes a coroutine task type, `std::execution::task` (`ex::task` from now on), to the C++ standard library. The paper first appeared early this year and was

initially discussed in Hagenberg, where it was forwarded by SG1, and received feedback in LEWG. The feature freeze deadline for C++26 then passed without a vote to forward it to LWG.

However, the paper was then further discussed in two LEWG telecons in April and May, where it was then forwarded to LWG for inclusion in C++26. This was done without following the procedure for making a schedule exception outlined in [P1000R6], and without taking an electronic vote to confirm the telecon decision.

In Sofia, LWG surprisingly found time to approve the wording, so it appeared in the Sofia plenary. After objections, the vote passed 77 to 11, with 29 abstentions, thus adding it to the C++26 standard.

I championed the plenary objections: First off, the proper procedure was not followed: The paper was forwarded after the feature freeze deadline in a telecon. Second, `ex::task` is an important feature that came late in the C++26 cycle to begin with, and I'm worried about technical issues in the design we might not be aware of yet. By the time they surface, C++26 is frozen, and any fixes are constrained by backwards compatibility. Third, I have concrete technical concerns with the design. As the committee plenary is not the appropriate venue for technical discussions, I am voicing them in this paper. I apologize for them coming late in the cycle; I did not realize until shortly before Sofia that `ex::task` was on track for C++26 and though I had the entire C++29 cycle to write a paper.

The author of [P3552R3] is also working on a paper collecting some issues and suggestions for fixes in [P3796R0].

§ 3 Major concerns

In my opinion, these issues need to be addressed before adopting `ex::task`. Unfortunately, I do not yet see a clear path to fix them in the limited time frame we have left for C++26.

§ 3.1 Iterative code can stack overflow

Consider:

```
ex::task<void> f(int i);

ex::task<void> g(int total) {
    for (auto i = 0; i < total; ++i) {
        co_await f(i);
    }
}
```

[Full example](#)

Depending on the value of `total` and the scheduler used to execute `g` on, this can lead to a stack overflow. Concretely, if the `ex::inline_scheduler` is used, each call to `f` will

execute eagerly, but, because `ex::task` does not support symmetric transfer, each schedule back-and-forth will add additional stack frames.

Having iterative code that is actually recursive is a potential security vulnerability.

A potential fix is adding symmetric transfer to `ex::task` with an operator `co_await` overload. However, while this would solve the example above, it would not solve the general problem of stack overflow when awaiting other senders. A thorough fix is non-trivial and requires support for guaranteed tail calls.

§ 3.2 Parameter lifetime is surprising

Consider:

```
struct Tracker {
    const char* label;

    Tracker(const char* label) : label(label) { std::printf("Tracker '%s'
        created\n", label); }
    Tracker(Tracker&& other) noexcept : label(other.label) { other.label =
        nullptr; }
    ~Tracker() {
        if (label != nullptr)
            std::printf("Tracker '%s' destroyed\n", label);
    }
};

ex::task<void> f(Tracker param = Tracker("f param")) {
    Tracker local("f local");
    co_return;
}

int main() {
    Tracker main("main");
    ex::sync_wait(f() | ex::then([] { std::printf("then\n"); }));
}
```

Full example

Unlike for a regular function call, where the return statement destroys the local variables, then the parameters, then returns to the caller, an `ex::task` function destroys the local variables, completes the resumer, triggering potentially arbitrary work, and only then destroys the parameters once the entire pipeline of senders is completed.

This is surprising behavior that can lead to unnecessary memory consumption and potentially hard to figure out bugs. It fundamentally breaks the promises of RAII where destruction is strictly tight to the end of a scope.

This is notably not an inherent flaw of coroutines, only of the particular design of `ex::task`. In particular, it is caused by the lifetime of `ex::task`'s operation state, see also [\[P3373R1\]](#).

§ 3.3 No protection against dangling references

(In-)Famously, reference parameters to coroutine functions are problematic because they are shallowly copied into the coroutine state: The caller has to make sure that the reference will live as long as the coroutine is running.

```
ex::task<int> f(const int& x);

ex::task<void> g() {
    auto task = f(11); // coroutine contains dangling reference
    co_await co; // error: dangling reference is observed
}
```

The core guidelines therefore recommends against passing parameters by reference (and they should also extend that to reference types such as `std::string_view`), and to avoid capturing lambdas as coroutines. The natural sender/receiver solution is structured concurrency, which ensures references live long enough. However, these are just guidelines, no enforced rules. Given the current climate, we should strive to have rules that are enforced by the compiler.

While the general solution requires extensive language changes, which is a bit unfair to expect from a library extension, Google's coroutine library [\[google-coro\]](#) has a pure library solution: Their default coroutine type `Co` is immovable and `co_await` takes it by-value. That way you can only `co_await` prvalue coroutines, which makes it impossible to have dangling references: All temporaries are guaranteed to live for the entire statement, which includes the `co_await` expression. They then have a separate movable `Future` type that packages all reference arguments together with the coroutine state.

```
Co<int> f(const int& x);

Co<void> g() {
    auto a = co_await f(42); // okay, temporary destroyed after the
                          `co_await`

    auto co = f(11); // coroutine contains dangling reference
    co_await co; // error: cannot co_await, so `f` cannot observe dangling
               reference
    co_await std::move(co); // likewise an error
}
```

This design is strictly safer than the current design of `ex::task`, but it would require some work to make it compatible with senders/receivers.

§ 4 Minor concerns

These are not issues that necessarily need to be fixed before standardization (nor do they ever need to be fixed), but if we didn't have the time pressure for C++26, I would have liked to see them addressed. Some of them could also be addressed in a future standard with a backwards-compatible evolution of the design.

§ 4.1 `co_yield with_error(x)` is clunky

The syntax to call `set_error` on the receiver without having to throw an exception is clunky:

```
ex::task<void> f() {  
    co_yield with_error(error_code);  
}
```

It would be much nicer to have `co_return` instead of `co_yield`: `co_return` has the usual semantics of returning a value and exiting a function; `co_yield` has the usual semantics of yielding a value and having the function continue later.

The reason `co_yield` is used, is that a coroutine promise can only specify `return_void` or `return_value`, but not both. If we want to allow `co_return`, we cannot have `co_return with_error(error_code);`. This is unfortunate, but could be fixed by changing the language to drop that restriction.

§ 4.2 `co_await ex::schedule(sch)` is an expensive no-op

`ex::task` is (correctly) scheduler affine; it will always be resumed on the same scheduler it was started on (unless the scheduler was explicitly changed). This leads to a potential footgun, however.

When using senders injecting `ex::schedule(sch)` means that future work is scheduled on `sch`:

```
ex::schedule(sch)  
| ex::then([]{ std::printf("on new scheduler\n"); })
```

Translating this naively to `ex::task`, `co_await ex::schedule(sch)` does nothing due to scheduler affinity of task:

```
ex::task<void> f() {  
    co_await ex::schedule(sch);  
    std::printf("still on old scheduler\n");  
}
```

The `co_await ex::schedule(sch)` schedules work on `sch`, but all the work does is reschedule the task back to the original scheduler: It is an expensive no-op.

The correct way to change the scheduler is to use `co_await ex::change_coroutine_scheduler(sch)`:

```
ex::task<void> f() {  
    co_await ex::change_coroutine_scheduler(sch);  
    std::printf("now on new scheduler\n");  
}
```

It is unfortunate that the nicer name (`ex::schedule`) is not what you want, and instead you have to use the more ugly `ex::change_coroutine_scheduler`. However, making `ex::schedule` do something special from other senders is also not a good idea, because then `co_await ex::schedule(sch)` and (potentially) `co_await (ex::schedule(sch) | ex::then([]{}))` would have different semantics, which is very confusing.

If we had the language change to make `co_return with_error(error_code)` work, `co_yield` is freed up and could to be used for scheduler changes. This is consistent with the use of `std::this_thread::yield` to yield the current thread back to the OS scheduler. Concretely, we could do a scheduler change with `co_yield sch`; (“yield to the scheduler `sch`”) and also introduce support for `co_yield`; with the semantics of yielding to the current scheduler. That way, at least the correct alternative to the wrong `co_await ex::schedule(sch)` is the shorter `co_yield sch`;

§ 4.3 Coroutine cancellation is ad-hoc

Senders have first-class support for cancellation with `set_stopped`, but coroutines do not. Therefore, `std::execution` has invented a custom customization point on the promise type, `unhandled_stopped` to allow propagation and recovery from cancellation.

A cleaner solution would extend language coroutines to support cancellation natively. It can also make it possible to have symmetric transfer when recovering from cancellation, instead of adding additional stack frames.

§ 5 Next steps

The design of [\[P3552R3\]](#) is not perfect; the author acknowledges that. However, they and a majority of WG21 believe that it is better to have something now, than nothing at all and that any obvious issues can be fixed using the NB comments process.

I fundamentally disagree.

As a standardization committee, we are drafting a standard that should outlive us. We are not working on some open-source library, we are designing the foundation for an entire ecosystem. If something has issues, and we know that it has issues, we should not have allowed a vote to approve it.

The authors did a great job in the limited time frame they had, and I thank them for it. However, if they feel its essential for `ex::task` to be in C++26, they should have written a paper earlier.

Trying to fix a design under time pressure before the standard is shipped out is a bad idea. I strongly urge the committee to reconsider the adoption of [\[P3552R3\]](#) for C++26.

§ 6 References

[google-coro] C++ Coroutines at Scale - Implementation Choices at Google - Aaron Jacobs - C++Now 2024.

<https://www.youtube.com/watch?v=k-A12dpMYHo>

[P1000R6] Herb Sutter. 2024-05-19. C++ IS schedule.

<https://wg21.link/p1000r6>

[P3373R1] Robert Leahy. 2025-01-12. Of Operation States and Their Lifetimes.

<https://wg21.link/p3373r1>

[P3552R3] Add a Coroutine Task Type.

<https://wg21.link/P3552R3>

[P3796R0] Coroutine Task Issues.

<https://wg21.link/P3796R0>